§6.AK Gate-Wiring Integration Patch — exact, copy-pasteable

Date: 2026-06-26 **Status:** Ready to apply (logic proven hermetically — see §6 below) **Closes:** §6.AK-debt (the mechanical wiring of `scripts/check-cycle-coverage.sh` into the distribute gate + pre-push hook). **Companion test:** `tests/cycle-coverage/test_wiring.sh` (5/5 PASS against the real gate; mutation-rehearsal proven). **Spec parent:** `docs/superpowers/specs/2026-06-26-ak-cycle-coverage-spec.md` §5.3 + §5.4.

`Classification`: project-specific (the two insertion patches are Lava-file-specific; the gate-coverage-must-intersect-cycle-claims pattern is universal per §6.AK).

> **IMPORTANT — this document does NOT edit the live gate files.** It is the precise patch the main stream applies later. The two target files (`scripts/firebase-distribute.sh`, `.githooks/pre-push`) are pushed-through-pre-push concurrently by the main stream; editing them here would corrupt an in-flight gate. Apply these patches only when the main stream is quiescent.

---

0. Why these two insertion points

`scripts/check-cycle-coverage.sh` already exists and passes its own hermetic test (`tests/cycle-coverage/test_cycle_coverage.sh`, 7/7). §6.AK-debt names exactly two unwired call-sites:

1. **`scripts/firebase-distribute.sh`** — a Phase-1 Gate that REFUSES the distribute at run time when the cycle-coverage gate fails (spec §5.3).
2. **`.githooks/pre-push`** — a Check that REJECTS a push which *advances a distribute `last-version` pointer* while the cycle-coverage gate fails (spec §5.4).

Both call the SAME committed gate with the SAME evidence file; they compose, they do not duplicate.

---

1. Patch A — `scripts/firebase-distribute.sh` (Phase-1 Gate 7)

1.1 Exact insertion point

Insert the new block **immediately after the Phase-1 Gates 4+5 block closes** and **before the `# 2. Resolve git SHA` section comment**. The anchor in the current file:

```
304 else
305     echo "    Phase 1 Gates 4+5 skipped: .env not present
        (auth feature inert at runtime; runtime falls back to
        StubLavaAuthBlobProvider)."
306 fi
307                         ← INSERT THE NEW BLOCK ON THIS BLANK
        LINE (between line 306 `fi` and line 308 comment)
308 # ------------------------------------------------------------------
309 # 2. Resolve git SHA + branch for the release notes
```

All variables the block references are already in scope at line 306: `MODE` (parsed @46–61), `SCRIPT_DIR` (@32), `APP_VERSION` / `APP_VERSION_CODE` (@113–116), `CHANGELOG_DIR` (@140, app-resolved). The block uses NO variable defined after line 306.

1.2 Block to paste (between line 306 and line 308)

```
# ------------------------------------------------------------------
# 1c. §6.AK Phase-1 Gate 7 — cycle-coverage (claims × executed
        device Challenges).
# Closes the firebase-distribute portion of §6.AK-debt. Refuses to
        distribute
# unless EVERY CHANGELOG-claimed user-visible fix for this version
        has an
# EXECUTED+PASSED covering device Challenge in the §6.Z evidence
        file for the
# SAME commit SHA (spec §5.3). This is the gate that would have
        CAUGHT the 1076
# incident (commit 627a0d58): a C00-only device gate while the
        CHANGELOG claimed
# search / provider-selection / onboarding fixes.
#
# NOTE: the gate's default --evidence-dir for the debug channel
        resolves to
# .../firebase-app-distribution-dev, but the client+api-app
        distribute layout
# keeps the §6.Z evidence in $CHANGELOG_DIR (firebase-app-
        distribution for
# client, -api-app for api-app). We therefore pass --evidence-
        dir="$CHANGELOG_DIR"
# explicitly. The cycle-coverage-map auto-resolves to
```

```
#   $CHANGELOG_DIR/${APP_VERSION}-${APP_VERSION_CODE}-cycle-
      coverage-map.yaml
# (the cycle author writes it alongside the §6.Z evidence).
#
# This block also subsumes the still-OPEN §6.Z runtime gate (§6.Z-
      debt): the
# cycle-coverage gate itself asserts evidence-presence + commit-
      SHA match +
# ≤24h freshness (exit 2 / exit 1) before checking claim coverage.
# ---------------------------------------------------------------
case "$MODE" in
    release) AK_CHANNEL="release" ;;
    *)       AK_CHANNEL="debug"   ;;
esac
echo "    Phase 1 Gate 7 (§6.AK): cycle-coverage — CHANGELOG
      claims × executed device Challenges"
ak_rc=0
"$SCRIPT_DIR/check-cycle-coverage.sh" \
    --version="$APP_VERSION-$APP_VERSION_CODE" \
    --channel="$AK_CHANNEL" \
    --evidence-dir="$CHANGELOG_DIR" \
    --strict || ak_rc=$?
case "$ak_rc" in
    0) echo
       "    §6.AK gate PASS — all CHANGELOG claims covered by
       executed device Challenges" ;;
    1) echo "FATAL §6.AK: CHANGELOG claim(s) lack a covering
       executed+PASSED device Challenge for $APP_VERSION-
       $APP_VERSION_CODE." >&2
       echo "       Run the missing device Challenge(s) on the
       gate, OR strike the unverified claim(s) from CHANGELOG.md
       (§6.AK clause 6)." >&2
       exit 1 ;;
    2) echo "FATAL §6.AK: §6.Z evidence or cycle-coverage-map
       missing / stale / wrong-SHA for $APP_VERSION-
       $APP_VERSION_CODE." >&2
       echo "       Re-run the device gate on THIS commit and
       write the cycle-coverage-map before distributing." >&2
       exit 1 ;;
    *) echo
       "FATAL §6.AK: internal error in cycle-coverage scanner
       (rc=$ak_rc)." >&2
       exit 1 ;;
esac
```

1.3 `set -e` correctness note (load-bearing)

`firebase-distribute.sh` runs under `set -euo pipefail` (@30). The gate exits non-zero on a covered-claim failure; the `|| ak_rc=$?` idiom is what prevents `set -e` from aborting the script *before* the `case` can emit the §6.AK-specific FATAL message. Do **not** rewrite this as a bare `"$SCRIPT_DIR/check-cycle-coverage.sh" ...; case $?`

`in` — under `set -e` the bare call would abort on a non-zero exit and the operator would never see the §6.AK directive. (This is exactly the idiom the wiring test proves in case `firebase_coverage_fail`.)

1.4 Scope: applies to BOTH apps

The block uses `$CHANGELOG_DIR` (app-resolved in §0 of the script: `firebase-app-distribution` for `--app client`, `firebase-app-distribution-api-app` for `--app api-app`). It therefore gates **both** artifacts per §6.AK clause 7. Implication for the main stream: **api-app distributes now also require** a `${APP_VERSION}-${APP_VERSION_CODE}-test-evidence.{md,json}` + `${APP_VERSION}-${APP_VERSION_CODE}-cycle-coverage-map.yaml` under the api-app channel dir. If the first post-graft api-app distribute has no claims, ship a map with the mandatory C00/C01 minimum per spec §3.3 (or an empty-CHANGELOG cycle records zero feature claims — but the map MUST still exist, else exit 2). If the main stream wants to defer api-app, wrap the block in `if [[ "$SELECTED_APP" == "client" ]]; then … fi` and record the api-app deferral as an explicit §6.AK-debt sub-item — but the constitutionally-correct default is universal.

---

2. Patch B — `.githooks/pre-push` (Check 10)

2.1 Exact insertion point

Insert the new Check **immediately after Check 7's `for ptr … done` loop closes** and **before the `# ===== Check 9` comment**. The anchor in the current file:

```
191         fi
192       fi
193     done                              ← this `done` closes Check 7's `for ptr in last-version-debug last-version-release`
194                                       ← INSERT THE NEW CHECK 10 BLOCK ON THIS BLANK LINE (between 193 and 195)
195     # ===== Check 9: §11.4.18 / CM-SCRIPT-DOCS-SYNC — script doc sync =====
```

The block lives inside the per-SHA `for sha in $range; do … done` loop (which spans lines 58–261) and appends to the `violations` array exactly like every other Check. It reuses Check 7's proven advance-detection idiom (`new_val > old_val` via `git show "$sha^:…"`).

2.2 Block to paste (between line 193 and line 195)

```
    # ===== Check 10: §6.AK cycle-coverage on distribute-
    advancing commits =====
# Closes the pre-push portion of §6.AK-debt (companion to
    firebase-
# distribute.sh's Phase-1 Gate 7). When a commit ACTUALLY
    advances

    # last-version-debug or last-version-release (NEW > OLD —
    same detection as

    # Check 7), a pointer-advance signifies "ready to
    distribute"; the §6.AK
# cycle-coverage gate MUST then pass for that version: every
    CHANGELOG-claimed
# user-visible fix needs an executed+PASSED covering device
    Challenge in the
# §6.Z evidence file. Pushing the advance without covering
    device evidence is
# the 1076 incident shape (627a0d58: C00-only gate shipped
    broken flows).
#
# NOTE: this mirrors Check 7's CLIENT-channel scope (firebase-
    app-distribution).
# api-app pointer advances live under firebase-app-
    distribution-api-app and
# are gated at distribute time by firebase-distribute.sh's
    Gate 7; extending
# Check 10 to the api-app channel is an owed follow-up (track
    under §6.AK-debt).
ak_chan_dir=".lava-ci-evidence/distribute-changelog/firebase-
    app-distribution"
for ptr in last-version-debug last-version-release; do
  ak_ptr_path="${ak_chan_dir}/${ptr}"
  git diff-tree --no-commit-id --name-only -r "$sha" 2>/dev/
    null | grep -qF "$ak_ptr_path" || continue
  ak_old=$(git show "$sha^:$ak_ptr_path" 2>/dev/null | tr -d
    ' \n' || echo "0")
  ak_new=$(git show "$sha:$ak_ptr_path" 2>/dev/null | tr -d '
    \n' || echo "0")
  [[ "$ak_new" =~ ^[0-9]+$ && "$ak_old" =~ ^[0-9]+$ &&
    "$ak_new" -gt "$ak_old" ]] || continue
  case "$ptr" in
    last-version-debug)   ak_channel="debug" ;;
    last-version-release) ak_channel="release" ;;
    *)                    ak_channel="debug" ;;
  esac

    # Resolve versionName for this advance: per-version
    snapshot first
  # (firebase-distribute writes pointer + snapshot
    atomically), then fall
```

```
    # back to build.gradle.kts versionName at this commit.
    ak_vname=""
    ak_snapshot=$(ls "${ak_chan_dir}"/*-${ak_new}.md 2>/dev/
      null | grep -v '\-test-evidence\.md$' | head -1 || true)
    if [[ -n "$ak_snapshot" ]]; then
      ak_vname=$(basename "$ak_snapshot" -${ak_new}.md)
    else
      ak_vname=$(git show "$sha:app/build.gradle.kts" 2>/dev/
      null | grep -oE 'versionName[[:space:]]*=[[:space:]]*"[^"]
      +"' | head -1 | sed 's/.*"\([^"]*\)".*/\1/' || echo "")
    fi
    [[ -n "$ak_vname" ]] || continue
    if [[ -x ./scripts/check-cycle-coverage.sh ]]; then
      ak_rc=0
      ./scripts/check-cycle-coverage.sh \
        --version="${ak_vname}-${ak_new}" \
        --channel="$ak_channel" \
        --evidence-dir="$ak_chan_dir" \
        --head="$sha" \
        --strict >/dev/null 2>&1 || ak_rc=$?
      if [[ "$ak_rc" -ne 0 ]]; then
        violations+=("$sha: §6.AK violation — advances $ptr $
      {ak_old}→${ak_new} (${ak_vname}) but the cycle-coverage
      gate exits $ak_rc. A CHANGELOG claim lacks a covering
      executed+PASSED device Challenge (or the §6.Z evidence /
      cycle-coverage-map is missing / stale / wrong-SHA). Run
      the missing device Challenge(s) on the gate, OR strike the
      unverified claim(s) from CHANGELOG.md (§6.AK clause 6),
      before pushing the pointer advance.")
      fi
    fi
  done
```

2.3 SHA-tie caveat (must be understood before applying)

Check 10 passes `--head="$sha"` (the advancing commit). `check-cycle-coverage.sh` therefore requires the §6.Z evidence file's `cycle-coverage:` header `commit=` to equal that advancing commit (else exit 2 → violation). This matches firebase-distribute.sh's existing **atomic** pointer+snapshot write (@484–512): the cycle author records the §6.Z evidence (with the build commit) in the **same** commit that advances the pointer. If a future workflow advances the pointer in a *separate* later commit from the one that wrote the evidence, the evidence `commit=` must be amended to that pointer-advancing commit, or Check 10 will (correctly, per §6.AK) reject the push. The `violations[]` message names this so the operator can act.

2.4 §6.N.1.2 + Check 9 obligations on the GRAFT COMMIT (do not skip)

The commit that applies BOTH patches will itself trip the existing hook:

- **Check 4 / §6.N.1.2** fires because the diff touches gate-shaping files (`.githooks/pre-push` AND `scripts/firebase-distribute.sh`, both in Check 4's list @97–98). The graft commit body MUST carry a `Bluff-Audit:` stamp that NAMES at least one file in the diff. Cite `tests/cycle-coverage/test_wiring.sh` as the rehearsal. Example stamp:

  ```
  Bluff-Audit: .githooks/pre-push + scripts/firebase-
  distribute.sh (§6.AK Check 10 + Gate 7)
    Mutation: swallow the §6.AK refusal (firebase `exit 1` → no-
  op; pre-push `violations+=` → no-op)
    Observed-Failure: tests/cycle-coverage/test_wiring.sh reports
      "FAIL [firebase_coverage_fail] exit=0 (expected 1)" and
      "FAIL [prepush_advance_fail] exit=0 (expected 1)" → RESULT:
  FAIL (exit 1);
      the firebase stub even prints "distribute would proceed"
  after a failing gate.
    Reverted: yes
  ```

- **Check 9 / CM-SCRIPT-DOCS-SYNC** fires because `docs/scripts/firebase-distribute.sh.md` already EXISTS — so the graft commit MUST also update that doc in the SAME commit (add a short "Phase 1 Gate 7 (§6.AK)" subsection). `.githooks/pre-push` is not under `scripts/*.sh`, so Check 9 does NOT apply to it.
- **Owed gap (flag, not blocking):** `docs/scripts/check-cycle-coverage.sh.md` is ABSENT. Check 9 only fires when `check-cycle-coverage.sh` is *modified* (we don't modify it here), so it does not block this graft — but the companion doc for that script is owed under CM-SCRIPT-DOCS-SYNC and should be authored next.

---

3. Evidence-file + map contract the cycle author must satisfy

For the Gate-7 / Check-10 pair to PASS on a real distribute, the cycle author writes two files under the channel dir (`.lava-ci-evidence/distribute-changelog/firebase-app-distribution/` for client):

1. **`<vname>-<code>-test-evidence.md`** (the §6.Z device-gate evidence) containing:
   - exactly one header line: `cycle-coverage: version=<vname>-<code> commit=<sha> channel=<debug|release> timestamp=<ISO-8601 ≤24h>`
   - one `challenge:` line per executed Challenge: `challenge: fqn=<FQN> verdict=<PASS|FAIL|SKIP> runner=<containers-submodule|genymotion-vm|…>` (a covering Challenge counts only when `verdict=PASS` AND `runner != host-direct`, per §6.AH).
2. **`<vname>-<code>-cycle-coverage-map.yaml`** mapping each CHANGELOG user-visible bullet to its `covering_challenge` (spec §3.1).

These are the same two file kinds the sibling gate test (`tests/cycle-coverage/test_cycle_coverage.sh`) and the wiring test (`tests/cycle-coverage/test_wiring.sh`) synthesize.

---

4. Apply order (recommended)

1. Verify the main stream is NOT mid-push (the two target files are pushed through pre-push concurrently).
2. Apply Patch A (firebase-distribute.sh) + update `docs/scripts/firebase-distribute.sh.md`.
3. Apply Patch B (.githooks/pre-push).
4. Re-run `bash tests/cycle-coverage/test_wiring.sh` (must stay 5/5) and `bash tests/cycle-coverage/test_cycle_coverage.sh` (must stay 7/7).
5. Commit with the §3.4-style `Bluff-Audit:` stamp naming the gate-shaping files.

---

5. What this does NOT change

- `scripts/check-cycle-coverage.sh` — untouched (already committed + tested).
- `tests/cycle-coverage/test_cycle_coverage.sh` — untouched.
- The §6.P / §6.AA / §6.Z gates already in firebase-distribute.sh — untouched; Gate 7 is additive and runs after them.

- Checks 1–9 in pre-push — untouched; Check 10 is additive.

---

6. Proof the wiring logic is sound (hermetic, no real files edited)

`tests/cycle-coverage/test_wiring.sh` generates STUB wrappers whose grafted blocks are byte-identical to §1.2 and §2.2, invokes the REAL `scripts/check-cycle-coverage.sh` against synthetic §6.Z evidence + map fixtures (HEAD + now injected via `LAVA_CYCLE_COVERAGE_HEAD` / `LAVA_CYCLE_COVERAGE_NOW_EPOCH`), and asserts:

```
test: §6.AK GATE-WIRING contract (firebase-distribute Phase-1 Gate
+ pre-push Check 10)
  PASS  [firebase_coverage_pass] exit=0 (expected 0) + matched
'§6.AK gate PASS'
  PASS  [firebase_coverage_fail] exit=1 (expected 1) + matched
'FATAL §6.AK'
  PASS  [prepush_advance_pass] exit=0 (expected 0) + matched
'Check 10 OK'
  PASS  [prepush_advance_fail] exit=1 (expected 1) + matched
'§6.AK violation'
  PASS  [prepush_no_advance] exit=0 (expected 0) + matched 'Check
10 OK'
------------------------------------------------------------
cases passed: 5   cases failed: 0
RESULT: PASS — firebase Phase-1 Gate + pre-push Check 10 wiring
proven against the real gate
```

Mutation rehearsal (swallow the refusal): the two negative cases flip to FAIL and the firebase stub prints "distribute would proceed" after a failing gate — the exact §6.AK incident shape the wiring exists to prevent. The wiring is therefore falsifiable and sound.